

Inventory Management System

Team Implementation Guide

Architecture: Controller -> Service -> Repository -> EF Core DbContext -> SQL Server

Category

Model: Id, Name, Description?, Products (collection)

- Create: Name(required), Description?
- Update: Id, Name, Description?
- Response: Id, Name, Description, ProductCount
- Repo: generic CRUD only
- Service: CRUD; prevent delete if has products
- Controller: GET /api/categories, GET /{id}, POST, PUT, DELETE

Product

Model: Id, Name, Description?, Price, Stock, CategoryId, Category, PurchaseProducts, SaleProducts

- Create: Name(req), Description?, Price(>0), Stock(>=0), CategoryId
- Update: Id, Name, Description?, Price, Stock, CategoryId
- Response: Id, Name, Description?, Price, Stock, CategoryId, CategoryName
- Repo: GetByCategoryAsync, SearchByNameAsync
- Service: CRUD + stock mgmt; validate category exists; prevent negative stock
- Controller: GET /api/products, GET /{id}, POST, PUT, DELETE, GET /category/{catId}, GET /search?name=...

Customer

Model: Id, Name, Email?, Phone?, Address?, Sales (collection)

- Create: Name(req), Email?, Phone?, Address?
- Update: Id, Name, Email?, Phone?, Address?
- Response: Id, Name, Email, Phone, Address
- Repo: SearchByNameAsync, GetByEmailAsync
- Service: CRUD; validate unique email
- Controller: GET /api/customers, GET /{id}, POST, PUT, DELETE, GET /search?name=...

Supplier

Model: Id, Name, Email?, Phone?, Address?

- Create: Name(req), Email?, Phone?, Address?
- Update: Id, Name, Email?, Phone?, Address?
- Response: Id, Name, Email, Phone, Address
- Repo: SearchByNameAsync
- Service: CRUD
- Controller: GET /api/suppliers, GET /{id}, POST, PUT, DELETE, GET /search?name=...

PurchaseProduct

Model: Id, ProductId, Quantity, DateIn, Notes?, Product

- Create: ProductId, Quantity(>0), DateIn, Notes?
- Update: Id, ProductId, Quantity, DateIn, Notes?
- Response: Id, ProductId, ProductName, Quantity, DateIn, Notes
- Repo: GetByProductAsync, GetByDateRangeAsync
- Service: CRUD + auto-increment stock on create; decrement stock on delete
- Controller: GET /api/purchase-products, GET /{id}, POST, PUT, DELETE, GET /product/{prodId}, GET /date-range?from=...&to=...

SaleProduct

Model: Id, ProductId, Quantity, DateOut, Notes?, Product

- Create: ProductId, Quantity(>0), DateOut, Notes?
- Update: Id, ProductId, Quantity, DateOut, Notes?
- Response: Id, ProductId, ProductName, Quantity, DateOut, Notes
- Repo: GetByProductAsync, GetByDateRangeAsync
- Service: CRUD + validate sufficient stock; auto-decrement stock; revert on delete/update
- Controller: GET /api/sale-products, GET /{id}, POST, PUT, DELETE, GET /product/{prodId}, GET /date-range?from=...&to=...

Sale

Model: Id, CustomerId, Date, TotalPrice, Status?, Customer

- Create: CustomerId, Date, Status?, line items (SaleProducts)
- Update: Id, CustomerId, Date, Status
- Response: Id, CustomerId, CustomerName, Date, TotalPrice, Status, SaleProducts
- Repo: GetByCustomerAsync, GetByDateRangeAsync
- Service: CRUD + auto-calculate TotalPrice from line items; update stock
- Controller: GET /api/sales, GET /{id}, POST, PUT, DELETE, GET /customer/{custId}, GET /date-range?from=...&to=...

Program.cs - DI Registration

```
builder.Services.AddScoped<ICategoryRepository, CategoryRepository>();
builder.Services.AddScoped<IProductRepository, ProductRepository>();
builder.Services.AddScoped<ICustomerRepository, CustomerRepository>();
builder.Services.AddScoped<ISupplierRepository, SupplierRepository>();
builder.Services.AddScoped<IPurchaseProductRepository, PurchaseProductRepository>();
builder.Services.AddScoped<ISaleProductRepository, SaleProductRepository>();
builder.Services.AddScoped<ISaleRepository, SaleRepository>();

builder.Services.AddScoped<ICategoryService, CategoryService>();
builder.Services.AddScoped<IProductService, ProductService>();
builder.Services.AddScoped<ICustomerService, CustomerService>();
builder.Services.AddScoped<ISupplierService, SupplierService>();
builder.Services.AddScoped<IPurchaseProductService, PurchaseProductService>();
builder.Services.AddScoped<ISaleProductService, SaleProductService>();
builder.Services.AddScoped<ISaleService, SaleService>();
```

Database Relationships

Category 1 ----* Product
Product 1 ----* PurchaseProduct
Product 1 ----* SaleProduct
Customer 1 ----* Sale

Naming Conventions

- Tables: snake_case (categories, products, purchase_products, etc.)
- Columns: PascalCase (C#) -> mapped via Fluent API
- Endpoints: kebab-case (/api/purchase-products, /api/sale-products)
- Namespaces: file-scoped (namespace X.Y.Z;)
- No cascading deletes - all FKs use DeleteBehavior.NoAction